

PROGRAM SKELETON

```
ORG 100h

.DATA
A   DB ?           ; 8-bit var
B   DW ?           ; 16-bit var
ARR DB 5 DUP(0)    ; array of 5
STR  DB 'Hello$'   ; $ = end
MSG  DB 13,10,'$'  ; newline

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX      ; init DS!

    ; ---- your code ----

    MOV AH, 4Ch      ; exit
    INT 21h
MAIN ENDP
END MAIN
```

DATA TYPES

DB = 8-bit byte (0--255)
DW = 16-bit word (0--65535)
DUP(n) = repeat n times
'text\$' = string for INT 21h/9

REGISTER MAP

Reg	Bits	Purpose
AX	16	Accumulator
AL/AH	8	Low/High of AX
BX	16	Base register
CX	16	Loop counter
DX	16	I/O, mul/div
SI	16	Array index
DI	16	Dest index
DS	16	Data segment

MUL 8-bit: AL × op → AX
MUL 16-bit: AX × op → DX:AX
DIV 8-bit: AX ÷ op → AL (quot), AH (rem)

DATA TRANSFER

```
; immediate -> register
MOV AX, 5
MOV AL, 'A'
MOV BL, 0FFh

; reg -> reg
MOV BX, AX
MOV DL, AL

; memory <-> register
MOV AL, A           ; var A
MOV AL, ARR[SI]     ; array

; load address
LEA DX, MSG
MOV DX, OFFSET MSG
```

```
; exchange
XCHG AL, BL
XCHG AX, BX

; clear register (fast)
XOR AX, AX
XOR SI, SI

; zero-extend AL to AX
MOV AH, 0
```

ARITHMETIC

```
; Addition / Subtraction
ADD AX, BX
ADD AL, 10
SUB AX, BX
SUB AL, 5
INC AX      ; AX++
DEC CX      ; CX--
NEG AL      ; AL = -AL

; Multiply (result in AX)
MOV AL, 5
MOV BL, 4
MUL BL      ; AX = 20

; Square: n*n
MOV AL, 7
MOV BL, AL
MUL BL      ; AX = 49

; Divide AX / BL
; AL=quotient AH=remainder
MOV AX, 20
MOV BL, 6
DIV BL      ; AL=3, AH=2
```

WARNING: Always MOV AH,0 before DIV when dividing an 8-bit number! Else AH has garbage.

LOGIC INSTRUCTIONS

```
AND AL, BL      ; bitwise AND
OR  AL, BL       ; bitwise OR
XOR AL, BL       ; bitwise XOR
NOT AL           ; 1's complement

; Check bit 0 (even/odd)
TEST AL, 1       ; sets ZF
JZ  EVEN         ; ZF=1 -> even

; Shifts
SHL AX, 1        ; AX * 2
SHR AX, 1        ; AX / 2
SAR AL, 1        ; signed right
```

COMPARE & JUMPS

```
CMP AX, BX      ; flags only
CMP AL, 0
CMP AL, '$'
```

panelJumpCondition

JE / JZ	Equal / Zero
JNE / JNZ	Not equal
JG	Greater (signed)
JGE	Greater or equal
JL	Less (signed)
JLE	Less or equal
JA	Above (unsigned)
JB	Below (unsigned)
JC	Carry flag set
JMP	Unconditional

LOOPS

```
; LOOP uses CX, auto DEC
MOV CX, 5
TOP:
    ; code here
    LOOP TOP      ; CX--, jmp if CX !=0

; Manual loop (when CX busy)
MOV BL, 5
TOP:
    DEC BL
    JNZ TOP

; Loop until char = '$'
TOP:
    MOV AL, [SI]
    CMP AL, '$'
    JE  DONE
    INC SI
    JMP TOP
DONE:
```

NESTED LOOPS: PUSH CX before inner loop, POP CX after.

INT 21h — I/O

```
; Read char -> AL
MOV AH, 1
INT 21h

; Print char in DL
MOV AH, 2
MOV DL, 'A'
INT 21h

; Print string (ends $)
MOV AH, 9
LEA DX, MSG
INT 21h

; Newline (CR+LF)
MOV AH, 2
MOV DL, 0Dh      ; Carriage Return
INT 21h
MOV DL, 0Ah      ; Line Feed
INT 21h
```

```
; Exit program
MOV AH, 4Ch
INT 21h
```

```
JB NOT_LOWER
CMP AL, 'z'
JA NOT_LOWER
```

```
PRINT:
MOV AL, ARR[SI]
ADD AL, 48 ; num -> ASCII
MOV DL, AL
MOV AH, 2
INT 21h
MOV DL, ' ' ; space
INT 21h
INC SI
LOOP PRINT
```

panel**AH=**
Function

01h Read char -> AL
02h Print DL as char
09h Print string at DX
0Ah Buffered string input
4Ch Exit program

Is Digit? ('0'-'9')

```
CMP AL, '0'
JB NOT_DIGIT
CMP AL, '9'
JA NOT_DIGIT
; is digit here
```

Find Largest in Array

```
MOV AL, ARR[0] ; assume max
MOV CX, 4 ; check rest
MOV SI, 1
LMAX:
CMP ARR[SI], AL
JLE SKIP
MOV AL, ARR[SI] ; new max
SKIP:
INC SI
LOOP LMAX
; AL = largest
```

ASCII & CONVERSIONS

panel	Operation	Code	Example
Char -> digit	SUB AL, 48	'5' -> 5	
Digit -> char	ADD AL, 48	5 -> '5'	
Lowercase -> Upper	SUB AL, 32	'a' -> 'A'	
Uppercase -> Lower	ADD AL, 32	'A' -> 'a'	
Toggle case	XOR AL, 20h	either	

Swap Two Variables

```
; Using XCHG
MOV AL, A
MOV BL, B
XCHG AL, BL
MOV A, AL
MOV B, BL

; Using temp
MOV AL, A
MOV TEMP, AL
MOV AL, B
MOV A, AL
MOV AL, TEMP
MOV B, AL
```

Find Smallest in Array

```
MOV AL, ARR[0]
MOV CX, 4
MOV SI, 1
LMIN:
CMP ARR[SI], AL
JGE SKIP
MOV AL, ARR[SI]
SKIP:
INC SI
LOOP LMIN
; AL = smallest
```

panel	Char range	ASCII	Hex
'0'--'9'	48-57	30h-39h	
'A'--'Z'	65-90	41h-5Ah	
'a'--'z'	97-122	61h-7Ah	
Space	32	20h	
CR	13	0Dh	
LF	10	0Ah	

Absolute Value

```
CMP AL, 0
JGE SKIP ; already positive
NEG AL ; negate if negative
SKIP:
```

N-th Letter from Input

```
; Input: AL=start char, BL=N
; Output: AL = char + N
MOV AH, 1
INT 21h ; char -> AL
MOV BL, 3 ; N = 3
ADD AL, BL ; e.g. 'B'+3='E'
MOV DL, AL
MOV AH, 2
INT 21h
```

Sum of Array

```
XOR AX, AX ; sum = 0
MOV CX, 5
MOV SI, 0
SUM:
MOV BL, ARR[SI]
MOV BH, 0
ADD AX, BX
INC SI
LOOP SUM
; AX = total sum
```

COMMON PATTERNS

Even / Odd Check

```
; Method 1: DIV
MOV AH, 0
MOV BL, 2
DIV BL
CMP AH, 0 ; AH = remainder
JZ EVEN
; else: ODD

; Method 2: TEST (faster)
TEST AL, 1
JZ EVEN ; bit0=0 -> even
JNZ ODD
```

Uppercase / Lowercase Check

```
; Is uppercase?
CMP AL, 'A'
JB NOT_UPPER
CMP AL, 'Z'
JA NOT_UPPER
; is uppercase here

; Is lowercase?
CMP AL, 'a'
```

ARRAY OPERATIONS

Input 5 digits into array

```
.DATA
ARR DB 5 DUP(0)
.CODE
MOV CX, 5
MOV SI, 0
READ:
MOV AH, 1
INT 21h
SUB AL, 48 ; ASCII -> num
MOV ARR[SI], AL
INC SI
LOOP READ
```

Print array elements

```
MOV CX, 5
MOV SI, 0
```

SORTING (BUBBLE SORT)

```
; Ascending bubble sort (N=5)
MOV CX, 4 ; outer: N-1
OUTER:
PUSH CX
MOV SI, 0
MOV CX, 4 ; inner: N-1
INNER:
MOV AL, ARR[SI]
MOV BL, ARR[SI+1]
CMP AL, BL
JLE NO_SWAP ; ascending
MOV ARR[SI], BL
MOV ARR[SI+1], AL
NO_SWAP:
INC SI
LOOP INNER
```

```

POP CX
LOOP OUTER
; For DESCENDING: change JLE to
JGE

```

```

DEC SI
LOOP REV2

```

```

ADD AL, '0'
HEXPRINT:
MOV DL, AL
MOV AH, 2
INT 21h

```

MATH ALGORITHMS

GCD (Euclidean)

```

; Input: AL=A, BL=B
; Output: AL = GCD
GCD PROC
    CMP BL, 0
    JE GCD_DONE
    MOV AH, 0
    DIV BL ; AH = A mod B
    MOV AL, BL
    MOV BL, AH
    CALL GCD
GCD_DONE:
    RET
GCD ENDP

```

PROCEDURES & MACROS

```

; -- Procedure definition --
NEWLINE PROC
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    RET
NEWLINE ENDP

PRINTCHAR PROC ; char in DL
    MOV AH, 2
    INT 21h
    RET
PRINTCHAR ENDP

```

```

; -- Call procedure --
CALL NEWLINE
MOV DL, 'X'
CALL PRINTCHAR

```

```

; -- Macro --
PRINT MACRO MSG
    MOV AH, 9
    LEA DX, MSG
    INT 21h
ENDM

```

```

PRINTLN MACRO MSG
    MOV AH, 9
    LEA DX, MSG
    INT 21h
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
ENDM

```

```

; -- Use macro --
PRINT MYMSG
PRINTLN HEADER

```

STACK USAGE

```

; Save/restore registers
PUSH AX
PUSH CX
PUSH SI
; ... code modifying AX,CX,SI ...
POP SI ; restore in REVERSE
POP CX
POP AX

```

```

; Nested loops need saved CX
MOV CX, 3 ; outer count
OUTER:
    PUSH CX
    MOV CX, 4 ; inner count
    INNER:
        ; inner code
    LOOP INNER
    POP CX ; restore outer CX

```

STRING OPERATIONS

String Length (till '\$')

```

MOV SI, OFFSET STR
XOR DL, DL ; count = 0
SLEN:
    MOV AL, [SI]
    CMP AL, '$'
    JE SLEN_END
    INC DL
    INC SI
    JMP SLEN
SLEN_END: ; DL = length

```

Count Char Occurrences

```

MOV AH, 1 ; input target
INT 21h
MOV BL, AL ; save it
MOV SI, OFFSET STR
MOV CX, LEN
XOR DL, DL
FREQ:
    MOV AL, [SI]
    CMP AL, BL
    JNE FSKIP
    INC DL
FSKIP:
    INC SI
    LOOP FREQ
; DL = frequency

```

LCM from GCD

```

; LCM = (A * B) / GCD(A,B)
MOV AL, A
MOV BL, B
CALL GCD ; AL = gcd
MOV CL, AL ; save gcd
MOV AL, A
MUL B ; AX = A*B
MOV BL, CL
DIV BL ; AL = LCM

```

Sum of 1..N

```

MOV CX, N
XOR AX, AX ; sum = 0
MOV BX, 1
SUMLOOP:
    ADD AX, BX
    INC BX
    LOOP SUMLOOP ; AX = N*(N+1)/2

```

Sum of Squares $1^2+2^2+...+N^2$

```

MOV CX, N
XOR BX, BX ; sum = 0
MOV AL, 1
SQ_LOOP:
    PUSH AX
    MUL AL ; AX = i*i
    ADD BX, AX
    POP AX
    INC AL
    LOOP SQ_LOOP ; BX = total

```

Average of Array

```

; (sum in AX, count in BL)
MOV AH, 0
DIV BL ; AL = average

```

Print HEX digit (0-F)

```

; AL = value 0-15
CMP AL, 10
JL ISDIGIT
ADD AL, 'A'-10 ; A-F
JMP HEXPRINT
ISDIGIT:

```

Reverse Print String

```

; Store input in buffer first
MOV CX, 5
MOV SI, 0
INP:
    MOV AH, 1
    INT 21h
    MOV BUF[SI], AL
    INC SI
    LOOP INP

; Print in reverse
MOV CX, 5
MOV SI, 4 ; last index
REV:
    MOV AL, BUF[SI]
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC SI
    LOOP REV

```

Reverse + Case Change

```

; lowercase input, UPPERCASE rev
MOV SI, 4
REV2:
    MOV AL, BUF[SI]
    SUB AL, 32 ; lower -> upper
    MOV DL, AL
    MOV AH, 2
    INT 21h

```

```
LOOP OUTER
```

LAB EXAM PROBLEM TEMPLATES

Prob 0: N-th char from input alphabet

```
; Input: char C, digit N
; Output: C + N (same case)
MOV AH, 1
INT 21h      ; C -> AL
MOV BL, AL   ; save char
MOV AH, 1
INT 21h      ; N -> AL
SUB AL, '0'   ; ASCII->num
ADD BL, AL    ; BL = C+N
MOV DL, BL
MOV AH, 2
INT 21h
```

Probs 1&2: Reverse + case change

```
; Read 5 chars into BUF
MOV CX, 5
MOV SI, 0
RD: MOV AH, 1 & INT 21h
    MOV BUF[SI], AL
    INC SI
    LOOP RD

; Print newline, then reverse
CALL NEWLINE
MOV CX, 5
MOV SI, 4
RV: MOV AL, BUF[SI]
    XOR AL, 20h    ; toggle case
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC SI
    LOOP RV
```

Probs 3&4: Even/Odd digits up to N

```
; Read N, print even digits 0..N
MOV AH, 1
INT 21h
SUB AL, '0'   ; N in AL
MOV BL, AL    ; limit
MOV AL, 0     ; start from 0
SCAN:
    CMP AL, BL
    JA DONE
    TEST AL, 1 ; check odd/even
    JNZ SKIP  ; JZ for ODD prob
    ; print AL as digit
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    SUB AL, '0' ; restore
SKIP:
    INC AL
    JMP SCAN
DONE:
```

Probs 5&6: Sort 5 input digits (asc/desc)

```
ORG 100h
.DATA
ARR DB 5 DUP(0)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV CX, 5      ; read 5 digits
    MOV SI, 0
READ:
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV ARR[SI], AL
    INC SI
    LOOP READ
    MOV CX, 4      ; bubble sort
outer
OUTER:
    PUSH CX
    MOV SI, 0
    MOV CX, 4
INNER:
    MOV AL, ARR[SI]
    MOV BL, ARR[SI+1]
    CMP AL, BL
    JLE NO_SW      ; JGE for
                    ; descending
    MOV ARR[SI], BL
    MOV ARR[SI+1], AL
NO_SW:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER
    MOV AH, 2      ; print newline
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    MOV CX, 5      ; print sorted
    MOV SI, 0
PRNT:
    MOV AL, ARR[SI]
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    INC SI
    LOOP PRNT
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Prob 7: Separate odd & even from 5 digits

```
ORG 100h
.DATA
ARR DB 5 DUP(0)
ODD DB 5 DUP(0)
EVEN DB 5 DUP(0)
MSG DB 'Odd: $'
EMSG DB 'Even: $'
.CODE
MAIN PROC
    MOV AX, @DATA
```

```
MOV DS, AX
MOV CX, 5
MOV SI, 0
RD:
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV ARR[SI], AL
    INC SI
    LOOP RD
    XOR BX, BX      ; odd index
    XOR DI, DI      ; even index
    MOV CX, 5
    MOV SI, 0
SEP:
    MOV AL, ARR[SI]
    TEST AL, 1
    JZ IS_E
    MOV ODD[BX], AL
    INC BX
    JMP NXT
IS_E:
    MOV EVEN[DI], AL
    INC DI
NXT:
    INC SI
    LOOP SEP
    MOV AH, 9      ; print "Odd: "
    LEA DX, MSG
    INT 21h
    MOV CX, BX      ; BX = odd
                    ; count
    MOV SI, 0
PO:
    MOV AL, ODD[SI]
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    INC SI
    LOOP PO
    MOV AH, 2      ; newline
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    MOV AH, 9      ; print "Even: "
    MOV DL, ' '
    LEA DX, EMSG
    INT 21h
    MOV CX, DI      ; DI = even
                    ; count
    MOV SI, 0
PE:
    MOV AL, EVEN[SI]
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    INC SI
    LOOP PE
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Probs 8–11: Sort odd/even subset (8 digits)

```

ORG 100h
.DATA
ARR DB 8 DUP(0)
SUB DB 8 DUP(0)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV CX, 8          ; read 8 digits
    MOV SI, 0

RD:
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV ARR[SI], AL
    INC SI
    LOOP RD
    XOR BX, BX          ; SUB[] count
    MOV CX, 8
    MOV SI, 0

FILT:
    MOV AL, ARR[SI]
    TEST AL, 1          ; JZ for even
                        probs
    JZ SKIP              ; JNZ for odd
                        probs
    MOV SUB[BX], AL
    INC BX

SKIP:
    INC SI
    LOOP FILT
    MOV DX, BX          ; save count
    CMP BX, 1          ; need >=2 to
                        sort
    JLE PRTONLY
    DEC BX              ; outer = count
                        -1

SOUT:
    PUSH BX
    MOV SI, 0
    MOV CX, BX

SINN:
    MOV AL, SUB[SI]
    MOV AH, SUB[SI+1]
    CMP AL, AH
    JLE NSW              ; JGE for
                        descending
    MOV SUB[SI], AH
    MOV SUB[SI+1], AL

NSW:
    INC SI
    LOOP SINN
    POP BX
    DEC BX
    JNZ SOUT

PRTONLY:
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    MOV CX, DX          ; print SUB[]
    MOV SI, 0

PRT:
    MOV AL, SUB[SI]
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    INC SI

```

```

    LOOP PRT
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN

```

Probs 12–14: Prime digits (8 input digits)

```

ORG 100h
.DATA
ARR DB 8 DUP(0)
PRMS DB 8 DUP(0)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV CX, 8
    MOV SI, 0

RD:
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV ARR[SI], AL
    INC SI
    LOOP RD
    XOR BX, BX          ; prime count
    MOV CX, 8
    MOV SI, 0

CHK:
    MOV AL, ARR[SI]
    CALL ISPRM
    CMP AH, 0
    JE SKIP
    MOV PRMS[BX], AL
    INC BX

SKIP:
    INC SI
    LOOP CHK
    MOV DX, BX          ; sort
                        ascending
    CMP BX, 1
    JLE PRT
    DEC BX

SOUT:
    PUSH BX
    MOV SI, 0
    MOV CX, BX

SINN:
    MOV AL, PRMS[SI]
    MOV AH, PRMS[SI+1]
    CMP AL, AH
    JLE NSW              ; JGE for
                        descend
    MOV PRMS[SI], AH
    MOV PRMS[SI+1], AL

NSW:
    INC SI
    LOOP SINN
    POP BX
    DEC BX
    JNZ SOUT

PRT:
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    MOV CX, DX          ; print SUB[]
    MOV SI, 0

SHOW:
    MOV AL, PRMS[SI]
    ADD AL, '0'

```

```

    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV DL, ' '
    INT 21h
    INC SI
    LOOP SHOW
    MOV AH, 4Ch
    INT 21h
MAIN ENDP

ISPRM PROC          ; input AL, out
                    AH
    MOV AH, 0
    CMP AL, 2
    JE YP
    CMP AL, 3
    JE YP
    CMP AL, 5
    JE YP
    CMP AL, 7
    JE YP
    RET
YP:
    MOV AH, 1
    RET
ISPRM ENDP
END MAIN

```

Prob 17: GCD of two digits

```

    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV [NUM1], AL
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV BL, AL
    MOV AL, [NUM1]
    CALL GCD          ; AL = gcd
    ADD AL, '0'
    MOV DL, AL
    MOV AH, 2
    INT 21h

```

Prob 18: Sum of 5 digits in HEX

```

ORG 100h
.DATA
MSG DB 'Sum: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    XOR BX, BX          ; sum = 0
    MOV CX, 5

SUMS:
    MOV AH, 1
    INT 21h
    SUB AL, '0'
    MOV AH, 0
    ADD BX, AX
    LOOP SUMS
    MOV AH, 9
    LEA DX, MSG
    INT 21h
    MOV AL, BL          ; print BL as
                        hex
    CMP AL, 10
    JL ISDIG
    ADD AL, 'A' - 10

```

```
JMP HPRT
ISDIG:
    ADD AL, '0'
HPRT:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Prob 19: String + Reverse 10 times each

```
ORG 100h
.DATA
STR DB 'Islamic University of
      Technology$'
SLEN EQU 32
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV CX, 10
RPT:
    PUSH CX
    MOV AH, 9 ; print forward
    LEA DX, STR
    INT 21h
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    MOV CX, SLEN ; print reverse
    MOV SI, SLEN-1
REVP:
    MOV AL, STR[SI]
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC SI
    LOOP REVP
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h
    POP CX
    LOOP RPT
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Prob 20: All ASCII in reverse order

```
ORG 100h
```

```
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AL, 0FFh ; start from
                255
A20:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC AL
    CMP AL, 0
    JNZ A20
    MOV DL, 0 ; print char 0
    MOV AH, 2
    INT 21h
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Prob 21: Digits+Alpha forward & reverse

```
ORG 100h
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AL, '0' ; forward:
                digits
A21F:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    INC AL
    CMP AL, '9'+1
    JL A21F
    MOV AL, 'A' ; forward:
                uppercase
A21U:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    INC AL
    CMP AL, 'Z'+1
    JL A21U
    MOV AL, 'a' ; forward:
                lowercase
A21L:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    INC AL
    CMP AL, 'z'+1
    JL A21L
    MOV AH, 2 ; newline
    MOV DL, 0Dh
    INT 21h
```

```
MOV DL, 0Ah
INT 21h
MOV AL, 'z' ; reverse:
            lowercase
R21L:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC AL
    CMP AL, 'a'-1
    JG R21L
    MOV AL, 'Z' ; reverse:
                uppercase
R21U:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC AL
    CMP AL, 'A'-1
    JG R21U
    MOV AL, '9' ; reverse:
                digits
R21D:
    MOV DL, AL
    MOV AH, 2
    INT 21h
    DEC AL
    CMP AL, '0'-1
    JG R21D
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

QUICK REFERENCE CARD

ALWAYS REMEMBER ORDER:
1. MOV AX,@DATA / MOV DS,AX
2. Read input (INT 21h, AH=1)
3. Process (loops / math / logic)
4. Print output (INT 21h, AH=2/9)
5. Exit (MOV AH,4Ch / INT 21h)

Pattern	Key instruction
Even/Odd	TEST AL,1 or DIV BL
Case toggle	XOR AL,20h
Char->num	SUB AL,48
Num->char	ADD AL,48
Clear reg	XOR AX,AX
Array idx	MOV AL,ARR[SI]
String end	CMP AL,'\$'
N-th char	ADD AL,N
Swap	XCHG AL,BL
Abs value	CMP+NEG
Sort	Bubble sort + XCHG
GCD	Euclid: A mod B

PRINT DIGIT 0-9:
ADD AL,48 / MOV DL,AL / MOV AH,2 / INT 21h
PRINT HEX 0-F:
CMP AL,10 / JL +3 / ADD AL,'A'-10 / JMP +2 / ADD AL,'0'
NEWLINE:
MOV DL,0Dh / MOV AH,2 / INT 21h / MOV DL,0Ah / INT 21h